structured bindings (с C++17)

Structured bindings — это механизм распаковки кортежей (и других агрегатов) в отдельные переменные

```
std::tuple<int, double, std::string> getData() {
    return {15, 8.7, "test"};
}

int main() {
    // распаковываем tuple
    auto [a, b, c] = getData();

    std::cout << a << std::endl;  // 15
    std::cout << b << std::endl;  // 8.7
    std::cout << c << std::endl;  // test
}
```

```
struct Point {
    int x; int y;
};

Point getPoint() {
    return {1, 2};
}

int main (){
    auto [x, y] = getPoint();
    std::cout << x << ", " << y; // 1, 2
}
```

Пример со своим типом

```
class MyClass {
    char* str;
    int value;
public:
    //конструкторы, деструктор и оператор =

    bool operator<(const MyClass& other) const {
        return strcmp(str,other.str) < 0;
    }
    bool operator==(const MyClass& other) const {
        return strcmp(str,other.str) == 0;
    }
    friend std::ostream& operator<< (std::ostream& o, const MyClass& t) {
        return o << t.value << ' ' << t.str << '\n';
    }
};
int main () {
    auto t1 = std::make_tuple(MyClass("Alice", 1),  MyClass("Mary",2));
    auto [a, b] = t1;
    std::cout << a; //Alice 1
}
```

Свёртка (Fold expression). Начиная с C++17

— это механизм для использования внутри функций/методов для обработки пакетов параметров без рекурсии

```
struct Group {
    int a = 5;
    const char* ch = "最近怎么样?\n";
};

std::ostream& operator << (std::ostream& out, const Group& gr) {
    return out << gr.a << ' ' << gr.ch;
}
template <typename... Types>
void print (const Types&... types) {
    (std::cout << ... << types );
}

int main () {
    Group group;
    print ("Hello", 5, 'A’);
    print (4.6, group);
}
```

```
((((((std::cout << "Hello") << 5) << 'A') << 4.6 ) << group.a << ' ' << group.ch);
```

Примеры

```
// Сумма всех аргументов
template<typename... Args>
auto sum(Args... args) {
    return (args + ...);  // унарная правая свёртка
}

// Все аргументы истинны?
template<typename... Args>
bool all_true(Args... args) {
    return (args && ...);  // логическое И
}

// Печать через пробел
template<typename... Args>
void print_space(Args... args) {
    ((std::cout << args << ' '), ...);
}

// Печать с разделителем
template<typename... Args>
void print_sep(char sep, Args... args) {
    ((std::cout << args << sep), ...);
}
```

Пример (наследование)

```
template<typename... Bases> // variadic
class Derived : public Bases... {
public:
    Derived(const Bases&... bases) : Bases(bases)... {}

    void printAll() const {
        ((Bases::print()), ...);  // свёртка
    }
};


struct A { void print() const { std::cout << "A\n"; } };
struct B { void print() const { std::cout << "B\n"; } };
struct C { void print() const { std::cout << "C\n"; } };


int main() {
    Derived<A, B, C> d(A{}, B{}, C{});
    d.printAll();  // A B C
}
```

Обработка исключений. Идиома RAII

Исключения

Механизм исключений (**exceptions**):

- Генерация сообщения об ошибке (throw).
- Перехват этих сообщений (catch).
- try блок для заключения одной или нескольких инструкций, которые <u>могут вызвать исключение</u>.

Генерация исключений

В языке C++ оператор throw используется для сигнализирования о возникновении исключения или ошибки

```
throw -1; // генерация исключения типа int

throw ENUM_INVALID_INDEX; // генерация исключения
                          // типа enum

throw "Can not take square root of negative number"; // генерация
                                                     // исключения типа
                                                     // const char* (строка C-style)
throw dX; // генерация исключения типа
          // double (переменная типа double,
         // которая была определена ранее)
throw MyException("Fatal Error"); // генерация исключения с использованием
                                  // объекта ранее определённого класса MyException
```

Компилятор гарантирует создание **специального временного объекта — объекта исключения**. Этот объект создается в специальном месте (не на обычном стеке, чтобы он не разрушился при раскрутке стека)

Объект исключения живет до тех пор, пока не будет пойман соответствующим catch

Поиск исключений

Блок try действует как наблюдатель в поисках исключений, которые были выброшены каким-либо из операторов в этом же блоке try, например:

```
try { // Здесь мы пишем стейтменты, которые будут генерировать следующее исключение
      throw -1; // типичный стейтмент throw
}
```

Обработка исключений

Ключевое слово catch используется для определения блока кода (так называемого "блока catch"), который обрабатывает исключения определенного типа данных:

```
catch (int a) { // Обрабатываем исключение типа int
       std::cerr << "We caught an int exception with value" << a << '\n';
}
```

При передаче объекта оператору throw блок catch получает копию этого объекта. И эта копия существует только в пределах блока catch. А можно без копирования?

Обработка исключений

```
#include <exception>

int divide (int a, int b) {
    if (b == 0) {
        throw std::logic_error("Not allowed");
    }
    return a/b;
}
int main () {
    divide(4,0);
}
```

В момент выброса исключения уничтожаются все локальные переменные функции, которая выбросила исключение, далее происходит свёртка стека до того момента, когда это исключение будет обработано.

Если исключение не обработать, программа всё равно завершится аварийно (terminating). В таком случае не гарантируется корректный вызов деструкторов.

try - catch

```
#include <exception>

int divide (int a, int b) {
    if (b == 0) {
        throw std::logic_error("Not allowed");
    }
    return a/b;
}
int main () {
    try {
        divide(4,0);
    } catch (std::logic_error& err) {
        std::cout << err.what();
    }
}
```

```
try {
    divide(4,0);
} catch (...) {
    std::cout << "Error";
}
```

Можно так

std::terminate()

Эта функция вызывается, если было выброшено исключение, но не обработано (подробнее см. cppreference).

Вызывает функцию abort() или функцию, заданную с помощью set_terminate, а также выводит ошибку, почему программа прервана.

Если исключение выброшено, не было обработано, то не гарантируется вызов деструкторов, т. е. возникает неопределённое поведение.

Свёртка стека при catch

- **Свёртка стека** (stack unwinding): вызываются деструкторы для всех созданных объектов в самой функции и в блоке try, как если бы они вышли из своей области видимости.

- **Механизм свёртки стека гарантирует**, что деструкторы для всех созданных автоматических объектов или полей класса в любом случае будут вызваны

Исключения стандартной библиотеки

В б**азовом классе** std::exception перечислены основные группы ошибок. Иерархия классов позволяет писать обработчик catch сразу на эту группу ошибок: std::logic_error, std::runtime_error и т. д. Например:

- Потоки ввода-вывода могут генерировать исключение std::ios_base::failure.
- Функция at у контейнеров std::array, std::vector и std::deque генерирует исключение std::out_of_range при некорректном индексе

std::exception

Defined in header <exception>

```
class exception;
```

Provides consistent interface to handle errors through the throw expression.

All exceptions generated by the standard library inherit from `std::exception`.

All member functions of `std::exception` are `constexpr`. (since C++26)

Member functions

(constructor)	constructs the exception object (public member function)
(destructor) [virtual]	destroys the exception object (virtual public member function)
operator=	copies exception object (public member function)
what [virtual]	returns an explanatory string (virtual public member function)

Standard exception requirements

Each standard library class T that derives from `std::exception` has the following publicly accessible member functions, each of them do not exit with an exception(until C++11) having a non-throwing exception specification(since C++11):

- default constructor (unless other constructors are provided)
- copy constructor
- copy assignment operator

The copy constructor and the copy assignment operator meet the following postcondition:

- If two objects `lhs` and `rhs` both have dynamic type T and `lhs` is a copy of `rhs`, then `std::strcmp(lhs.what(), rhs.what())` is equal to `0`.

The `what()` member function of each such T satisfies the constraints specified for `std::exception::what()`.

Standard exceptions

- `logic_error`
  - `invalid_argument`
  - `domain_error`
  - `length_error`
  - `out_of_range`
  - `future_error` (since C++11)
- `runtime_error`
  - `range_error`
  - `overflow_error`
  - `underflow_error`

?

```
int main () {
    try {
        throw 1;
    } catch (double d) {
        std::cout << "double";
    } catch (long d) {
        std::cout << "long";
    } catch (...) {
        std::cout << "other";
    }
}
```

```
int main () {
    try {
        throw 1;
    } catch (double d) {
        std::cout << "double";
    } catch (long d) {
        std::cout << "long";
    }
}
```

Слева будет other, справа не будет обработана ошибка

Правила catch

Если происходит исключение, для него подбирается подходящий catch-обработчик. Таких обработчиков может быть несколько. Выбирается первый подходящий по типу обработчик и выполняется его код. Если же ни один обработчик не подходит по типу, то исключение считается необработанным. В этом случае оно пробрасывается дальше по стеку — туда, откуда была вызвана текущая функция. А если обработчик не найдётся даже в функции main, то программа аварийно завершается (вызывается std::terminate()).

Когда выполняется catch, не применяется правило перегрузки и не выполняется конверсия.

Но! Можно обрабатывать ссылку родительского класса, как ошибку наследника, а также через «срез».

Обработка ошибок при «срезе»

```
class Base {};
class Derived: public Base {};

int main () {
    try {
        Derived d;
        throw d;
    } catch (Base) {
        std::cout << "Base";
    } catch (Derived) {
        std::cout << "Derived";
    } catch (...) {
        std::cout << "other";
    }
}
```

Будет выведено Base

```
class Base {};
class Derived: Base {};

int main () {
    try {
        Derived d;
        throw d;
    } catch (Base) {
        std::cout << "Base";
    } catch (Derived) {
        std::cout << "Derived";
    } catch (...) {
        std::cout << "other";
    }
}
```

Будет выведено Derived

Гарантии безопасности исключений

- **Гарантия отсутствия сбоев**. Функции с такими гарантиями вообще не выбрасывают исключений.
- **Строгая гарантия безопасности**. Исключение может возникнуть, но от этого объект нашего класса не поменяет состояние: количество элементов останется прежним, итераторы и ссылки не будут инвалидированы и т. д.
- **Базовая гарантия безопасности**. При исключении состояние объекта может поменяться, но оно останется внутренне согласованным, то есть, инварианты будут соблюдаться.
- **Отсутствие гарантий**. Это довольно опасная категория: при возникновении исключений могут нарушаться инварианты

RAII (Resource Acquisition Is Initialization)

RAII (Получение ресурса есть инициализация) — это фундаментальная идиома C++, которая связывает время жизни ресурса с временем жизни объекта. Ресурс захватывается в конструкторе и освобождается в деструкторе.

```
class Point {
private:
    double x;
    double y;
    char* name;
public:
    Point(double x_coord = 0.0, double y_coord = 0.0, const char* point_name = nullptr)
        : x(x_coord), y(y_coord), name(nullptr) {
            name = new char[std::strlen(point_name) + 1];
            std::strcpy(name, point_name);}
    Point(const Point& other): x(other.x), y(other.y), name(nullptr) {
            name = new char[std::strlen(other.name) + 1];
            std::strcpy(name, other.name); }
    ~Point() { delete[] name; }
    Point& operator=(const Point& other) {
        Point temp(other);
        swap(temp);
        return *this; }
    void swap(Point& other) {
        std::swap(x, other.x);
        std::swap(y, other.y);
        std::swap(name, other.name); }
};
```

Идиома RAII. Постановка задачи

```
void g(int y){
    if (y==0) {
        throw 1;
    }
}

void f(int x) {
    int* p = new int(x);
    g(*p);
    delete p;
}

int main () {
    f(5);
}.
```

Если в функции g(*p) будет брошено исключение, выделенная память не будет освобождена.

Необходимо, конечно, обрабатывать try-catch, но можно просто забыть освободить ресурс

```
void g(int* p){
    if (*p==0) {
       throw 1;
    }
}
void f(int x) {
   int * p = new int[x];
   try{
      g(p);
   }
   catch (...) {
      std::cout << "Error";
      delete [] p;
   }
}

int main () {
    f(0);
}
```

Возможное решение. Оформить некоторый класс, который будет освобождать память за нас.

```
struct Wrapper {
    int* p;
    Wrapper(int* p): p(p) {}
    ~Wrapper() { delete p;}
    int& operator* () {
        return *p;
    }
};

void g(Wrapper y){
    if (*y==0) {
        throw 1;
    }
}
void f(int x) {
    Wrapper w(new int(x));
    g(w);
}
int main () {
    f(5);  //Указатель копируется, попытка освобождения ранее освобожденной памяти
}
```

Решение. Resource Acquisition Is Initialization
Получение ресурса есть инициализация

```
template <typename T>
struct unique_ptr {
    T* p;
    unique_ptr(T* p): p(p) {}
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator= (const unique_ptr&) = delete;
    ~unique_ptr() { delete p;}
    T& operator* () {
        return *p;
    }
};
void g(unique_ptr<int>& y){
    if (*y==0) {
        throw 1;
    }
}
void f(int x) {
    unique_ptr<int> p(new int(x));
    g(p);
}
int main () {
    f(5);
}
```

Здесь приведён пример структуры, которая позволяет освобождать память в случае возникновения ошибки. Чтобы избежать попытки освобождения ранее освобождённой памяти, запретим копировать и присваивать элементы структуры. Также, чтобы такой указатель мог быть любого типа, сделаем эту структуру шаблонной.

Обратите внимание!!!!
Если брошенную ошибку не обработать, даже тут не гарантируется вызов деструктора!

Умные указатели

Умный указатель (англ. *smart pointer*) — это объект, работать с которым можно как с обычным указателем, но при этом, в отличии от последнего, он предоставляет дополнительный функционал (автоматическое освобождение закрепленной за указателем области памяти и не только).

Три вида: unique_ptr, shared_ptr и weak_ptr

Умный указатель "владеет" ресурсом: память будет освобождена в его деструкторе. Слово unique в его названии подчёркивает, что это единственный владелец ресурса.

Когда std::unique_ptr выходит из области видимости, утечки памяти не происходит, потому что в своем деструкторе умный указатель вызывает delete для объекта, на который ссылается, высвобождая тем самым память.

Важно понять, что внутри умные указатели всё равно используют new/delete, они лишь позволяют программисту не делать этого и, как следствие, защищают его от ошибок.

std::shared_ptr

std::shared_ptr – умный указатель с подсчётом ссылок на объект. **Такой указатель можно копировать**. При копировании увеличивается счётчик созданных копий. В деструкторе этот счётчик уменьшается.

Объект удаляется последним владельцем, когда счётчик дойдёт до нуля. Этот счётчик хранится в отдельной ячейке динамической памяти.

На неё ссылаются все объекты shared_ptr, которые разделяют владение одним и тем же объектом в динамической памяти. Текущее значение счётчика можно узнать с помощью функции use_count.

```cpp
#include <iostream>
#include <memory>

int main (){
    std::shared_ptr<int> ptr1 = std::make_shared<int>(17);
    std::cout << *ptr1 << "\n"; // 17
    std::cout << ptr1.use_count() << "\n"; // 1

    auto ptr2 = ptr1; // копирование разрешено!
    std::cout << *ptr1 << "\n"; // 17
    std::cout << *ptr2 << "\n"; // 17 — это всё тот же объект
    std::cout << ptr1.use_count() << "\n"; // 2
    std::cout << ptr2.use_count() << "\n"; // 2

    std::shared_ptr<int> ptr3;
    std::cout << ptr3.use_count() << "\n"; // 0

    ptr3 = ptr1; // присваивание тоже разрешено!
    std::cout << *ptr3 << "\n"; // 17
    std::cout << ptr1.use_count() << "\n"; // 3
    std::cout << ptr2.use_count() << "\n"; // 3
    std::cout << ptr3.use_count() << "\n"; // 3
}
```

```
...

int main() {
    std::shared_ptr<Awesome> ptr1 = std::shared_ptr<Awesome>(new Awesome());
    std::cout << ptr1.use_count() << "\n"; // 1
    {
        auto ptr2 = ptr1;
        std::cout << ptr2.use_count() << "\n"; // 2
    }
    std::cout << ptr1.use_count() << "\n"; // 1
}
```

Существуют ситуации, когда объект A должен ссылаться на B, а B – на A. Это называется циклической ссылкой (*cyclic reference/circular dependency*). В таком случае оба объекта никогда не будут выгружены из памяти.

Чтобы разорвать цикличность, необходимо использовать std::weak_ptr

std::weak_ptr

Всё дело в том, что объекты, которыми владеют std::shared_ptr уничтожаются, только когда не остается владельцев (тех самых shared_ptr). Если же родительский компонент владеет дочерним, а дочерний компонент содержит shared_ptr на родительский компонент, т.е. владеет родительским компонентом, то они не дают уничтожиться друг другу.

std::weak_ptr является умным указателем, который не владеет объектом.

Если родительский компонент поменять на std::weak_ptr, то родительский компонент теперь будет являться владельцем дочерних компонентов, а дочерние компоненты перестанут быть владельцами родительского.

```
std::vector<std::shared_ptr<Component>> children;

std::weak_ptr<Component> parent;
```

Правило трёх → Правило пяти

Правило пяти → Правило ноля

Правило ноля

Определено Мартино Фернандесом: “не определяйте самостоятельно ни одну из функций 5ки, вместо этого поручите заботу о владении ресурсами специально придуманным для этого классам”, то есть *std::unique_ptr, std::shared_ptr, std::weak_ptr*. Эти типы для управления ресурсами, в свою очередь, уже будут следовать Правилу Пяти.

Благодаря тому, что при использовании этих классов существует возможность задавать пользовательские *deleter*’ы, с помощью них можно реализовать большинство из стратегий владения памятью.

Правило Ноля заключается в том, что тот сценарий, когда *не нужно* определять *ничего* из специальных функций-членов, должен быть *предпочтительным*

И даже с полиморфными объектами

```
struct base {virtual void foo() = 0;};
struct derived : base {void foo() override {...}};


base* bad = new derived;
delete bad; // !!! Нет виртуального деструктора в base

{
    ...
    std::shared_ptr<base> good = std::make_shared<derived>();

} // shared_ptr при разрушении вызовет правильный деструктор.
```

Исключения в конструкторе

```
struct A {
    A() {std::cout << "A";}
    ~A() {std::cout << "~A";}
};
struct B {
    A a;
    B(int x) {
         std::cout << "B";
         if (x == 0) throw 1;
    }
    ~B() {
        std::cout << "~B";
    }
};
int main (){
    try {
        B b(0);
    } catch (...) {

    }
}
```

Лучше использовать умные указатели

```
class Inner {
public:
    Inner() { std::cout << "InnerCtor\n"; }
    ~Inner() { std::cout << "InnerDtor\n"; }
};
class Outer {
    Inner* raw_ptr;
    std::unique_ptr<Inner> smart_ptr;
public:
    Outer() : raw_ptr(new Inner()), smart_ptr(std::make_unique<Inner>()) {
        throw std::runtime_error("Ошибка в конструкторе");
    }
    ~Outer() {
        std::cout << "OuterDtor\n";
        delete raw_ptr;                    // НЕ удалится!
    }
};
int main () {
    try {
        Outer o;
    }
    catch (...){
    } // InnerCtor InnerCtor InnerDtor
}
```

Исключения в списке инициализации

```
struct A {
    A(int x) {
        if (x == 0) throw 1;
        std::cout << "A";}
    ~A() {std::cout << "~A";}
};
struct B {
    A a;
    A a1;
    B(int x) try: a(1), a1(0) {
         std::cout << "B";
    } catch (...){
         std::cout << "Error";
    }
    ~B() {
         std::cout << "~B";
    }
};
int main (){
    try {
         B b(0);
    } catch (...) {
    }
}
```

```
struct A {
    A(int x) {
        if (x == 0) throw 1;
        std::cout << "A";}
    ~A() {std::cout << "~A";}
};
struct B {
    A a;
    A a1;
    B(int x) try: a(1), a1(0) {
         std::cout << "B";
    } catch (...){
         std::cout << "Error";
    }
    ~B() {
         std::cout << "~B";
    }
};
int main (){
    B b(0);
}
```

terminating with uncaught exception
abort ./a.out

Исключение в деструкторе выбрасывать нельзя (потому что он сам может быть вызван по причине исключения)

```
struct A {
    A(int x) {}
    ~A() {std::cout << "~A";}
};
struct B {
    A a;
    B(int x): a(1) {
        std::cout << "B";
    }
    ~B() {
        std::cout << "~B";
        throw 2;
    }
};
int main (){
    try {
        B b(0);
    } catch (...) {
        std::cout << "Error";
    }
}
```

Несмотря на блок try-catch, ошибка обработана не будет. Если деструктор бросает исключение, вызывается сразу функция std::terminate()

abort ./a.out

Но можно бросать и сразу обрабатывать

```
struct A {
    A(int x) {std::cout << "A";}
    ~A() {std::cout << "~A";}
};
struct B {
    A a;
    B(int x): a(1) {
        std::cout << "B";
    }
    ~B() {
        std::cout << "~B";
        try {
          throw 2;
        } catch(...) {
           std::cout << "Error";
        }
    }
};
int main (){
    B b(0);
}
```

Вывод:

A

B

~B

Error

~A

Но если очень хочется…

Начиная с C++11, функцию можно пометить как noexcept, это говорит компилятору, что это функция не бросает исключения. А если в ней выброшено исключение, то оно там же и обработано.

```
void foo() noexcept try {

} catch (...) {
}
```

Если исключение было всё же брошено, то будет вызов std::terminate.

```
void foo() noexcept {
    throw 1;
}
```

warning: 'foo' has a non-throwing exception specification but can still throw

Условный noexcept

Можно установить noexcept от определённого условия:

```
template <typename T>
void foo() noexcept(std::is_reference_v<T>) {

}
```

Например, если T является ссылкой, то функция помечается noexcept

Так вот, деструктор

Деструкторы все неявно помечены noexcept, поэтому если хотим разрешить бросить исключение, то следует снять noexcept:

```
struct A {
    A(int x) {std::cout << "A";}
    ~A() {std::cout << "~A";}
};

struct B {
    A a;
    B(int x): a(1) {
        std::cout << "B";
    }
    ~B() noexcept(false) {
        std::cout << "~B";
        throw 2;
    }
};
```

```
int main (){
    try {
        B b(2);
    } catch (int& r) {
        std::cout << "Sorry";
    }
}
```

В таком случае ошибка будет обработана, программа завершится нормально. Вывод: A B ~B ~A Sorry